OS

Operating systems

Ten theory questions from OS fundamentals to virtual memory, with model answers, the key points examiners look for, and common mistakes. It pairs with the Memory Management Model Paper, which covers the numericals.

10 questions · 5 marks each · 50 marks · Suggested time: 90 min

Unit 1: OS basics and system calls

Q1. Role of the OS and dual-mode operation (5 marks)

What is an operating system? Explain its two main roles. (2)
What is dual-mode operation? Why is it needed? Give three examples of privileged instructions. (3)

Model answer

(a) An operating system is the software layer between applications and hardware. It manages the hardware and gives programs a simpler, safer way to use it. Its two roles:

Resource manager: it shares the CPU, memory, disks and devices among many programs fairly and efficiently, and keeps them from interfering with each other.
Abstraction provider (extended machine): it hides hardware details behind clean abstractions: processes instead of raw CPU, virtual address spaces instead of physical RAM, files instead of disk blocks.

(b) **Dual-mode operation** means the CPU runs in one of two modes, set by a **mode bit** in hardware. In **kernel mode** any instruction can run. In **user mode** privileged instructions are blocked; trying one causes a trap to the OS. User programs run in user mode and the OS kernel runs in kernel mode.

It is needed for **protection**: a buggy or malicious program must not be able to crash the system, read another process's memory, or take over a device. The only way into kernel mode is a controlled entry point (interrupt, trap or system call).

Privileged instructions: I/O instructions; setting the timer; enabling or disabling interrupts; changing the page table base register; switching the mode bit itself.

Key points for marks: two roles named and explained; mode bit; user programs cannot run privileged instructions; protection as the reason; three valid privileged instructions.

Common mistake: listing "reading a file" as a privileged instruction. It is a system call, which is how a user program asks the kernel to do privileged work on its behalf.

Q2. System calls, interrupts and traps (5 marks)

What is a system call? Describe, step by step, what happens when a program calls `read()`. (3)
Distinguish between an interrupt, a trap (exception) and a system call. (2)

Model answer

(a) A **system call** is the programming interface through which a user program requests a service from the kernel, such as file I/O, creating a process, or allocating memory.

The program calls the library wrapper `read()` (in libc).
The wrapper puts the system call number and arguments in registers.
It executes a special trap instruction (`syscall` on x86-64, `int 0x80` on older x86). The CPU switches to kernel mode.
The CPU jumps to the kernel's trap handler, at an address the OS set up at boot. The kernel saves the user registers.

The kernel looks up the call number in the system call table and runs the handler, which checks the arguments and does the work. The process may block if data is not ready.
The return value is placed in a register, user registers are restored, and a return-from-trap instruction switches back to user mode.

(b)

	Interrupt	Trap / exception	System call
Source	Hardware device (disk, timer, keyboard)	The running instruction (divide by zero, page fault, illegal instruction)	The program, deliberately
Timing	Asynchronous: any time	Synchronous: caused by the current instruction	Synchronous and intentional
Example	Disk finishes a read	Page fault	`read()`, `fork()`

All three switch the CPU to kernel mode and run an OS handler. A system call is a trap the program causes on purpose.

Key points for marks: definition; call number in a register; trap instruction and mode switch; system call table; return to user mode; synchronous vs asynchronous distinction.

Common mistake: saying the program "jumps to the kernel function". User code cannot jump to kernel addresses; it can only enter through the trap mechanism to a handler address the OS chose.

Unit 2: Processes and threads

Q3. Process, address space and PCB (5 marks)

Differentiate a program from a process. (1)
Draw and explain the memory layout of a process. (2)
What is a process control block (PCB)? List its main contents. (2)

Model answer

(a) A **program** is a passive file on disk: code and static data. A **process** is a program in execution: it has its own memory, CPU state and resources. One program can run as many processes.

(b) From low to high addresses:

Region	Holds	Grows?
Text (code)	Machine instructions, read-only	Fixed
Data	Global and static variables	Fixed

Heap	Dynamically allocated memory (`malloc`, `new`)	Upwards
Stack	Function frames: local variables, arguments, return addresses	Downwards

The heap and stack grow towards each other, leaving unused space between them.

(c) The **PCB** is the kernel's data structure for one process. It holds everything needed to stop and resume the process:

Process ID and parent ID
Process state (ready, running, blocked…)
Saved CPU registers and program counter
Memory information: page table pointer, memory limits
Scheduling information: priority, time used
Open files and I/O status

Key points for marks: passive vs active; four regions in order with growth directions; PCB as kernel structure; at least four PCB fields including saved registers.

Q4. Process states and context switching (5 marks)

Draw the process state diagram and explain each transition. (3)
What is a context switch? Why is it considered overhead? (2)

Model answer

States: New, Ready, Running, Blocked (Waiting), Terminated.

Transition	Cause
New → Ready	Process created and admitted by the OS
Ready → Running	Scheduler picks it (dispatch)
Running → Ready	Preempted: timer interrupt, or a higher-priority process arrives
Running → Blocked	Waits for I/O or an event (e.g. `read()`,`wait()`)
Blocked → Ready	The I/O or event completes
Running → Terminated	Calls `exit()` or is killed

A blocked process never goes straight to Running. It must return to Ready and be chosen by the scheduler.

(b) A **context switch** is the kernel saving the CPU state (registers, program counter, stack pointer) of the running process into its PCB, and loading the saved state of the next process. If the processes have different address spaces, the page table base register also changes, and the TLB may need flushing.

It is **overhead** because no user work is done during the switch, and afterwards caches and the TLB are "cold" for the new process, so it runs slower for a while.

Key points for marks: five states; all six transitions with causes; save and restore via PCB; address space change; no useful work during switch, cache/TLB effects.

Common mistake: confusing a **mode switch** (user to kernel, e.g. a system call, same process) with a **context switch** (one process to another). A system call does not always cause a context switch.

Q5. Process API: fork, exec, wait, exit (5 marks)

Explain what `fork()`, `exec()`, `wait()` and `exit()` do. (2)
What are zombie and orphan processes? How does the OS deal with each? (2)
Why does Unix use separate `fork()` and `exec()` calls instead of one "create process" call? (1)

Model answer

`fork()` creates a new child process that is a copy of the parent. It returns twice: 0 in the child and the child's PID in the parent (−1 on failure).
`exec()` replaces the current process's memory image with a new program. On success it does not return; the PID stays the same.
`wait()` blocks the parent until a child terminates, then collects the child's exit status.
`exit()` terminates the calling process, frees its resources and keeps its exit status for the parent.

(b) A **zombie** is a child that has exited but whose parent has not yet called `wait()`. Its PCB entry stays so the exit status can be collected. The parent reaps it by calling `wait()`. An**orphan** is a child whose parent exited first. The OS re-parents it to `init` (PID 1), which calls`wait()` on it when it exits.

(c) Between `fork()` and `exec()` the child can change its own environment before the new program starts. A shell uses this to set up I/O redirection and pipes (for example, closing standard output and opening a file before running `ls > out.txt`).

Key points for marks: fork returns twice with different values; exec replaces the image and does not return; wait reaps; zombie vs orphan and init; redirection as the reason.

Common mistake: saying `exec()` creates a new process. It reuses the same process (same PID) with a new program.

Q6. Threads (5 marks)

What is a thread? What do threads of a process share, and what does each keep private? (2)
Give two benefits of multithreading over using multiple processes. (1)
Compare user-level and kernel-level threads, and name the three threading models. (2)

Model answer

(a) A **thread** is a single sequence of execution within a process. A process can have several threads running in the same address space.

Shared by all threads	Private to each thread
Code, global data, heap, open files, address space	Thread ID, program counter, registers, stack

(b) Threads are cheaper to create and switch than processes, because no new address space is needed. They share memory directly, so communication needs no IPC. On multicore machines threads of one process can run in parallel.

(c)

	User-level threads	Kernel-level threads
Managed by	A library in user space; kernel does not know about them	The kernel
Switch cost	Very cheap, no system call	Needs a kernel entry
Blocking call	Blocks the whole process	Blocks only that thread
Multicore	Cannot run in parallel	Can run in parallel

Models: **many-to-one** (many user threads on one kernel thread), **one-to-one** (each user thread is a kernel thread; used by Linux and Windows), **many-to-many** (many user threads multiplexed on a smaller or equal number of kernel threads).

Key points for marks: stack and registers are private; heap and globals are shared; two benefits; blocking and parallelism differences; three models named.

Common mistake: saying threads have separate heaps. They share one heap; only the stack is per thread.

Unit 3: CPU scheduling

Q7. Scheduling criteria and preemption (5 marks)

Define the five common scheduling criteria and say whether each should be maximised or minimised. (2.5)
When does the CPU scheduler run? Use this to distinguish preemptive from non-preemptive scheduling. (2.5)

Model answer

Criterion	Meaning	Goal
CPU utilisation	Fraction of time the CPU is busy	Maximise
Throughput	Processes completed per unit time	Maximise
Turnaround time	Completion time − arrival time	Minimise
Waiting time	Total time spent in the ready queue	Minimise

Response time	Arrival until first run (first response)	Minimise

(b) The scheduler may run when a process:

switches from running to blocked (I/O request, `wait()`);
switches from running to ready (timer interrupt);
switches from blocked to ready (I/O completes);
terminates.

Non-preemptive scheduling runs only in cases 1 and 4: a process keeps the CPU until it blocks or finishes. **Preemptive** scheduling can also run in cases 2 and 3, taking the CPU away from a running process. Preemption gives better response time but needs care with shared data, since a process can be stopped at any point.

Key points for marks: five criteria with correct directions; four scheduling points; non-preemptive = cases 1 and 4 only; trade-off of preemption.

Common mistake: mixing up turnaround and waiting time. Turnaround includes running and I/O time; waiting time counts only time in the ready queue.

Q8. Comparing scheduling algorithms (5 marks)

What is the convoy effect? Which algorithm suffers from it? (1)
SJF is optimal for average waiting time. Why is it hard to use in practice, and what problem can it cause? (2)
In round robin, what happens if the time quantum is very large, or very small? (1)
What is starvation in priority scheduling, and how does aging solve it? (1)

Model answer

(a) The **convoy effect** is when many short processes wait behind one long CPU-bound process, like cars behind a slow truck. It raises average waiting time. FCFS suffers from it.

(b) SJF needs the length of the next CPU burst, which is not known in advance. The OS can only **predict** it, usually with an exponential average of past bursts: $\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$. SJF can also cause **starvation**: a long process may never run if short ones keep arriving.

(c) Very large quantum: round robin becomes FCFS. Very small quantum: too much time goes on context switches, so throughput drops. A common rule is that most CPU bursts (about 80%) should finish within one quantum.

(d) **Starvation**: a low-priority process waits forever because higher-priority processes keep arriving. **Aging** gradually raises the priority of processes that have waited a long time, so every process eventually runs.

Algorithm	Preemptive ?	Strength	Weakness
FCFS	No	Simple, fair by arrival	Convoy effect
SJF / SRTF	No / Yes	Lowest average waiting time	Burst length unknown; starvation
Round robin	Yes	Good response time	Quantum choice; higher turnaround

Priority	Either	Important work first	Starvation (fix: aging)

Key points for marks: convoy effect with FCFS; burst prediction; starvation; both quantum extremes; aging explained.

Unit 4: Memory management

Q9. Paging, segmentation and the TLB (5 marks)

Compare paging and segmentation on four points. (2)
Why is a TLB needed with paging? Explain how address translation uses it. (2)
Why are multi-level page tables used? (1)

Model answer

	Paging	Segmentation
Unit size	Fixed (pages)	Variable (segments)
Based on	Physical convenience; invisible to the programmer	Logical units: code, stack, heap
Fragmentation	Internal only	External only
Address	Page number + offset	Segment number + offset (checked against limit)

(b) The page table lives in memory, so every access would need two memory accesses: one for the page table entry, one for the data. The **TLB** is a small, fast hardware cache of recent page-to-frame translations. The MMU first looks up the page number in the TLB. On a hit, it forms the physical address at once. On a miss, it reads the page table, uses the entry, and stores it in the TLB for next time.

(c) A single-level table for a 32-bit space with 4 KB pages needs 2^{20} entries (4 MB) per process, even if most of the address space is unused. Multi-level tables create inner tables only for regions in use, saving memory.

Key points for marks: four contrasts with fragmentation type correct; two memory accesses without TLB; hit and miss paths; memory saving of multi-level tables.

Q10. Virtual memory and demand paging (5 marks)

What is virtual memory? Give two benefits. (2)
Explain demand paging and the role of the valid and present bits. (2)
What is thrashing? (1)

Model answer

(a) **Virtual memory** separates a process's logical address space from physical memory, so each process sees a large private address space, which can be bigger than physical RAM. Benefits: programs larger than RAM can run; more processes fit in memory at once (higher multiprogramming); processes are isolated from each other; memory can be shared (e.g. libraries) easily.

(b) **Demand paging** loads a page into RAM only when the process first accesses it; other pages stay on disk in swap space or the executable file. In the lecture's convention:

Valid bit: the page belongs to the process's address space. Access to an invalid page is a segmentation fault.
Present bit: the page is currently in RAM. Access to a valid but not-present page causes a page fault, and the OS loads the page from disk and restarts the instruction.

(c) **Thrashing** is when the system spends more time swapping pages in and out than doing useful work, because processes have fewer frames than their working sets need.

Key points for marks: definition and two benefits; load on first access; valid vs present meaning; page fault handling; thrashing linked to the working set.

Terminology note: Galvin uses a single valid–invalid bit, where "invalid" means either not in memory or not legal. Accept either convention if explained correctly.

Original questions written for this course. Fork/exec details follow Unix and Linux behaviour; scheduling criteria and models follow standard textbook definitions (Silberschatz, Galvin and Gagne; OSTEP).